

Principles of Software Design

Software Design is also a process to plan or convert the software requirements into a step that are needed to be carried out to develop a software system. There are several principles that are used to organize and arrange the structural components of Software design. Software Designs in which these principles are applied affect the content and the working process of the software from the beginning.

These principles are stated below :

1. **Should not suffer from “Tunnel Vision” –**

While designing the process, it should not suffer from “tunnel vision” which means that it should not only focus on completing or achieving the aim but on other effects also.

2. **Traceable to analysis model –**

The design process should be traceable to the analysis model which means it should satisfy all the requirements that software requires to develop a high-quality product.

3. **Should not “Reinvent The Wheel” –**

The design process should not reinvent the wheel that means it should not waste time or effort in creating things that already exist. Due to this, the overall development will get increased.

4. **Minimize Intellectual distance –**

The design process should reduce the gap between real-world problems and software solutions for that problem meaning it should simply minimize intellectual distance.

5. **Exhibit uniformity and integration –**

The design should display uniformity which means it should be uniform throughout the process without any change. Integration means it should mix or combine all parts of software i.e. subsystems into one system.

6. **Accommodate change –**

The software should be designed in such a way that it accommodates the change implying that the software should adjust to the change that is required to be done as per the user's need.

7. **Degrade gently –**

The software should be designed in such a way that it degrades gracefully which means it should work properly even if an error occurs during the execution.

8. **Assessed or quality –**

The design should be assessed or evaluated for the quality meaning that during the evaluation, the quality of the design needs to be checked and focused on.

9. **Review to discover errors –**

The design should be reviewed which means that the overall evaluation should be done to check if there is any error present or if it can be minimized.

10. **Design is not coding and coding is not design –**

Design means describing the logic of the program to solve any problem and coding is a type of language that is used for the implementation of a design.

Software Design Process – Software Engineering

The design phase of software development deals with transforming the customer requirements as described in the SRS documents into a form implementable using a programming language. The software design process can be divided into the following three levels or phases of design:

1. Interface Design
2. Architectural Design
3. Detailed Design

Elements of a System

1. **Architecture:** This is the conceptual model that defines the structure, behavior, and views of a system. We can use flowcharts to represent and illustrate the architecture.
2. **Modules:** These are components that handle one specific task in a system. A combination of the modules makes up the system.
3. **Components:** This provides a particular function or group of related functions. They are made up of modules.

4. **Interfaces:** This is the shared boundary across which the components of a system exchange information and relate.
5. **Data:** This is the management of the information and data flow.

Interface Design

Interface design is the specification of the interaction between a system and its environment. This phase proceeds at a high level of abstraction with respect to the inner workings of the system i.e, during interface design, the internal of the systems are completely ignored, and the system is treated as a black box. Attention is focused on the dialogue between the target system and the users, devices, and other systems with which it interacts. The design problem statement produced during the problem analysis step should identify the people, other systems, and devices which are collectively called agents.

Interface design should include the following details:

1. Precise description of events in the environment, or messages from agents to which the system must respond.
2. Precise description of the events or messages that the system must produce.
3. Specification of the data, and the formats of the data coming into and going out of the system.
4. Specification of the ordering and timing relationships between incoming events or messages, and outgoing events or outputs.

Architectural Design

Architectural design is the specification of the major components of a system, their responsibilities, properties, interfaces, and the relationships and interactions between them. In architectural design, the overall structure of the system is chosen, but the internal details of major components are ignored. Issues in architectural design includes:

1. Gross decomposition of the systems into major components.
2. Allocation of functional responsibilities to components.
3. Component Interfaces.
4. Component scaling and performance properties, resource consumption properties, reliability properties, and so forth.
5. Communication and interaction between components.

The architectural design adds important details ignored during the interface design. Design of the internals of the major components is ignored until the last phase of the design.

Detailed Design

Detailed design is the specification of the internal elements of all major system components, their properties, relationships, processing, and often their algorithms and the data structures. The detailed design may include:

1. Decomposition of major system components into program units.
2. Allocation of functional responsibilities to units.
3. User interfaces.
4. Unit states and state changes.
5. Data and control interaction between units.
6. Data packaging and implementation, including issues of scope and visibility of program elements.
7. Algorithms and data structures.

Objectives of Software Design

1. **Correctness:** A good design should be correct i.e., it should correctly implement all the functionalities of the system.
2. **Efficiency:** A good software design should address the resources, time, and cost optimization issues.
3. **Flexibility:** A good software design should have the ability to adapt and accommodate changes easily. It includes designing the software in a way, that allows for modifications, enhancements, and scalability without requiring significant rework or causing major disruptions to the existing functionality.
4. **Understandability:** A good design should be easily understandable, it should be modular, and all the modules are arranged in layers.
5. **Completeness:** The design should have all the components like data structures, modules, external interfaces, etc.
6. **Maintainability:** A good software design aims to create a system that is easy to understand, modify, and maintain over time. This involves using modular and well-structured design principles e.g.,(employing appropriate naming conventions and providing clear documentation). Maintainability in Software and design also enables developers to fix bugs, enhance features, and adapt the software to changing requirements without excessive effort or introducing new issues.

Software Design Concepts

Concepts are defined as a principal idea or invention that comes into our mind or in thought to understand something. The **software design concept** simply means the idea or principle behind the design. It describes how you plan to solve the problem of designing software, and the logic, or thinking behind how you will design software. It allows the software engineer to create the model of the system software or product that is to be developed or built. The software design concept provides a supporting and essential structure or model for developing the right software. There are many concepts of software design and some of them are given below:

Points to be Considered While Designing Software

1. **Abstraction (Hide Irrelevant data):** Abstraction simply means to hide the details to reduce complexity and increase efficiency or quality. Different levels of Abstraction are necessary and must be applied at each stage of the design process so that any error that is present can be removed to increase the efficiency of the software solution and to refine the software solution. The solution should be described in broad ways that cover a wide range of different things at a higher level of abstraction and a more detailed description of a solution of software should be given at the lower level of abstraction.
2. **Modularity (subdivide the system):** Modularity simply means dividing the system or project into smaller parts to reduce the complexity of the system or project. In the same way, modularity in design means subdividing a system into smaller parts so that these parts can be created independently and then use these parts in different systems to perform different functions. It is necessary to divide the software into components known as modules because nowadays, there are different software available like Monolithic software that is hard to grasp for software engineers. So, modularity in design has now become a trend and is also important. If the system contains fewer components then it would mean the system is complex which requires a lot of effort (cost) but if we can divide the system into components then the cost would be small.
3. **Architecture (design a structure of something):** Architecture simply means a technique to design a structure of something. Architecture in designing software is a concept that focuses on various elements and the data of the structure. These components interact with each other and use the data of the structure in architecture.
4. **Refinement (removes impurities):** Refinement simply means to refine something to remove any impurities if present and increase the quality. The refinement concept of software design is a process of developing or presenting the software or system in a detailed manner which means elaborating a system or software. Refinement is very necessary to find out any error if present and then to reduce it.
5. **Pattern (a Repeated form):** A pattern simply means a repeated form or design in which the same shape is repeated several times to form a pattern. The pattern in the design process means the repetition of a solution to a common recurring problem within a certain context.

6. **Information Hiding (Hide the Information):** Information hiding simply means to hide the information so that it cannot be accessed by an unwanted party. In software design, information hiding is achieved by designing the modules in a manner that the information gathered or contained in one module is hidden and can't be accessed by any other modules.
7. **Refactoring (Reconstruct something):** Refactoring simply means reconstructing something in such a way that it does not affect the behavior of any other features. Refactoring in software design means reconstructing the design to reduce complexity and simplify it without impacting the behavior or its functions. Fowler has defined refactoring as “the process of changing a software system in a way that it won't impact the behavior of the design and improves the internal structure”.

SOLID Principles Overview

SOLID is an acronym for five design principles intended to make software designs more understandable, flexible, and maintainable. These principles are particularly applicable in object-oriented programming.

1. Single Responsibility Principle (SRP)

Definition: A class should have only one reason to change, meaning it should only have one job or responsibility.

Benefits:

- Enhances code clarity and maintainability.
- Simplifies debugging and testing, as changes in one area won't impact unrelated functionality.

Before SRP:

```
python

class User:
    def __init__(self, name):
        self.name = name

    def save(self):
        # Logic to save user data
        pass

    def send_email(self, message):
        # Logic to send email
        pass
```

Class User: This class has multiple responsibilities:

- It holds user data (name).
- It contains a method to save user data.
- It has a method to send emails to the user.

After SRP:

```
python

class User:
    def __init__(self, name):
        self.name = name

class UserRepository:
    def save(self, user):
        # Logic to save user data
        pass

class EmailService:
    def send_email(self, user, message):
        # Logic to send email
        pass
```

Refactoring:

- The User class now only contains user data.
- A new class, UserRepository, is created to handle saving user data.
- Another class, EmailService, manages sending emails.

Benefits: Each class has a single responsibility, making it easier to manage, test, and modify.

2. Open/Closed Principle (OCP)

Definition: Software entities (classes, modules, functions, etc.) should be open for extension but closed for modification.

Benefits:

- Promotes code reusability.
- Minimizes the risk of introducing bugs when new features are added.

Example:

Before OCP:

```
python

class Shape:
    def area(self, shape):
        if isinstance(shape, Circle):
            return 3.14 * shape.radius ** 2
        elif isinstance(shape, Rectangle):
            return shape.width * shape.height
```

Class Shape: This method checks the type of shape and calculates the area accordingly.

Problem: Adding a new shape requires modifying the area method, violating OCP.

After OCP:

```
python

from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height
```

- **Refactoring:**
 - Shape is now an abstract base class with an abstract method area().
 - Circle and Rectangle implement the area() method specific to their shape.
- **Benefits:** New shapes can be added by creating new subclasses without modifying existing code.

3. Liskov Substitution Principle (LSP)

Definition: Subtypes must be substitutable for their base types without altering the correctness of the program.

Benefits:

- Ensures that a derived class can stand in for its base class.
- Facilitates code reuse and robustness.

Example:

Before LSP:

```
python

class Bird:
    def fly(self):
        return "Flying"

class Ostrich(Bird):
    def fly(self):
        raise Exception("Ostriches can't fly")
```

Class Bird: Has a method fly().

Class Ostrich: Inherits from Bird, but cannot fly. This violates LSP because it cannot substitute Bird without causing an error.

After LSP:

```
python

class Bird(ABC):
    @abstractmethod
    def move(self):
        pass

class Sparrow(Bird):
    def move(self):
        return "Flying"

class Ostrich(Bird):
    def move(self):
        return "Running"
```

Refactoring:

- Bird is now an abstract class with a move() method.
- Both Sparrow and Ostrich implement move() appropriately for their capabilities.

Benefits: Any subclass of Bird can be used interchangeably without breaking the code.

4. Interface Segregation Principle (ISP)

Definition: No client should be forced to depend on methods it does not use.

Benefits:

- Promotes a more modular design.
- Reduces side effects and dependencies.

Example:

Before ISP:

```
python

class MultiFunctionPrinter:
    def print(self, document):
        # Print logic
        pass

    def scan(self, document):
        # Scan logic
        pass

    def fax(self, document):
        # Fax logic
        pass
```

Class MultiFunctionPrinter: Implements multiple methods (print, scan, fax).

Problem: If a client only needs to print, it still depends on the other methods, violating ISP.

After ISP:

```
python

class Printer(ABC):
    @abstractmethod
    def print(self, document):
        pass

class Scanner(ABC):
    @abstractmethod
    def scan(self, document):
        pass

class Fax(ABC):
    @abstractmethod
    def fax(self, document):
        pass
```

Refactoring:

- Created separate interfaces for each functionality: Printer, Scanner, and Fax.

Benefits: Clients can implement only the interfaces they need, leading to cleaner, more focused implementations.

5. Dependency Inversion Principle (DIP)

Definition: High-level modules should not depend on low-level modules; both should depend on abstractions.

Benefits:

- Reduces tight coupling between components.
- Enhances testability and flexibility.

Example:

Before DIP:

```
python

class Database:
    def connect(self):
        # Connect to database
        pass

class UserService:
    def __init__(self):
        self.database = Database()
```

Class UserService: Directly depends on the concrete Database class, making it hard to switch databases.

After DIP:

```
python

from abc import ABC, abstractmethod

class Database(ABC):
    @abstractmethod
    def connect(self):
        pass

class MySQLDatabase(Database):
    def connect(self):
        # MySQL connection logic
        pass

class UserService:
    def __init__(self, database: Database):
        self.database = database
```

Refactoring:

- Introduced an abstract Database class.

- UserService now depends on the Database abstraction instead of a concrete implementation.

Benefits: Allows swapping of different database implementations without changing UserService, improving flexibility and testability.

BENEFITS:

1. Improved Maintainability

- **Easier to Update:** When each class or module has a single responsibility, changes in one area of the codebase are less likely to impact others. This isolation makes it easier to update and maintain the code.

2. Enhanced Flexibility

- **Easier to Extend:** The Open/Closed Principle allows for new functionality to be added with minimal changes to existing code. This flexibility helps adapt to changing requirements without extensive refactoring.

3. Reduced Complexity

- **Simplified Design:** By adhering to principles like the Single Responsibility Principle and Interface Segregation Principle, the design remains clean and modular. This reduces the cognitive load on developers, making it easier to understand and work with the code.

4. Better Code Reusability

- **Reusable Components:** The principles promote the creation of generic and reusable components. For instance, Dependency Inversion allows for easier swapping of implementations, which is beneficial in various contexts (e.g., different database systems).

5. Increased Testability

- **Easier to Unit Test:** With clear separation of concerns and interfaces, individual components can be tested in isolation. This leads to more reliable unit tests, making it easier to identify and fix bugs.

6. Enhanced Collaboration

- **Team Productivity:** In team environments, SOLID principles foster better collaboration. Team members can work on different components simultaneously without stepping on each other's toes, as the components have well-defined interfaces.

7. Higher Code Quality

- **Reduced Bugs:** Following these principles often leads to fewer bugs. Since components are well-defined and responsibilities are clear, the likelihood of unexpected behavior diminishes.

8. Alignment with Agile Practices

- **Agility and Responsiveness:** SOLID principles align well with agile development practices, allowing teams to quickly respond to changing requirements and deliver value incrementally.

Global Usage of SOLID Principles

1. **Software Development Companies**
 - o **ThoughtWorks:** Known for its focus on agile development and clean coding practices, ThoughtWorks implements SOLID principles in its software solutions across various industries.
2. **Agile Development Teams**
 - o **Spotify:** The music streaming giant employs agile methodologies and emphasizes SOLID principles to maintain the scalability and adaptability of its software.
3. **Open Source Projects**
 - o **Django:** This popular web framework for Python encourages the use of SOLID principles in its architecture, promoting clean and maintainable code.
4. **Web Development**
 - o **JetBrains:** The company behind IntelliJ IDEA and other development tools uses SOLID principles to ensure the maintainability of its complex software products.
5. **Mobile App Development**
 - o **Zynga:** This game development company applies SOLID principles in its mobile app development to manage complex game architectures and enhance performance.
6. **Cloud Computing**
 - o **Netflix:** As a leader in cloud-based services, Netflix employs SOLID principles in its microservices architecture to ensure that services are loosely coupled and scalable.

Usage of SOLID Principles in Pakistan

1. **Software Houses**
 - o **Techlogix:** This IT services company applies SOLID principles in developing custom enterprise solutions for various clients, ensuring high-quality software delivery.
2. **Startups**
 - o **Bykea:** A ride-hailing and logistics startup, Bykea uses SOLID principles in its app development to manage complexity and ensure scalability as it grows.
3. **Freelancers and Remote Teams**
 - o **Creative Chaos:** This digital agency based in Pakistan employs SOLID principles to deliver high-quality web and mobile solutions to clients around the world.

4. Education and Training

- o **Lahore University of Management Sciences (LUMS):** Their Computer Science program includes SOLID principles in the curriculum, preparing students for modern software development practices.

5. IT Consultancies

- o **Confiz:** This IT consultancy incorporates SOLID principles in its software development projects, offering training and implementation services to clients to improve their code quality.

CASE STUDIES:

Solid design principles, often referred to as SOLID principles, are five key object-oriented design guidelines that promote maintainability, scalability, and robustness in software design. They were introduced by Robert C. Martin (Uncle Bob) and have since been applied widely across various industries. Below are case studies that illustrate the application of these principles:

Single Responsibility Principle (SRP)

Case Study: Refactoring a Payroll System

Problem: The payroll system for a large company had a class responsible for calculating salaries, logging operations, and sending notifications. As the system grew, this class became complex and difficult to maintain.

Solution: The class was refactored into three classes: one for salary calculation, one for logging, and one for notifications. Each class had a single responsibility.

Outcome: This refactoring made the code more maintainable and allowed for easier updates or replacements of individual components without affecting the others.

Open/Closed Principle (OCP)

Case Study: E-commerce Discount Strategy

Problem: An e-commerce platform had a discount system that required modification every time a new discount type was added. This made the system brittle, as adding new discount types meant editing the existing code.

Solution: The system was refactored using inheritance and polymorphism, creating an abstract `Discount` class with different discount types (e.g., percentage, flat rate) implemented as subclasses. Now, new discount types could be added by extending the `Discount` class without modifying the existing code.

Outcome: The platform could now add new discount types without affecting the core logic, following the open/closed principle (open for extension, closed for modification).

Liskov Substitution Principle (LSP)

Case Study: Shapes Library

Problem: A geometric shapes library had a `Rectangle` class and a `Square` class, where the `Square` class was inheriting from `Rectangle`. However, since a square's width and height must always be equal, this violated the LSP when a `Square` object was substituted for a `Rectangle` object, leading to incorrect behavior.

Solution: The inheritance structure was re-evaluated, and instead of `Square` being a subclass of `Rectangle`, both classes inherited from a more general `Shape` class. This allowed both rectangles and squares to behave correctly when used in place of one another.

Outcome: By adhering to LSP, the library became more flexible and predictable.

Interface Segregation Principle (ISP)

Case Study: Printer System

Problem: A multifunctional printer interface had methods like `Print`, `Fax`, `Scan`, and `Photocopy`. However, not all printers could perform all these functions, forcing some printer classes to implement methods they didn't need, violating ISP.

Solution: The large interface was broken down into smaller, more specific interfaces, such as `IPrinter`, `IScanner`, and `IFax`. Now, printers could implement only the interfaces that matched their actual functionality.

Outcome: This made the system more flexible and allowed for easier implementation of new printer models without unnecessary dependencies.

Dependency Inversion Principle (DIP)

Case Study: Email Notification System

Problem: An email notification system had a class tightly coupled with the `SmtpClient` class. When the email sending mechanism needed to be changed (e.g., to an external service), it required substantial modification.

Solution: The email sending functionality was abstracted into an interface called `IEmailSender`, which could be implemented by various classes (`SmtpEmailSender`, `ExternalEmailSender`).

The main notification class now depended on the `ISender` abstraction, not the concrete implementation.

Outcome: This allowed the system to switch between different email providers without modifying the core logic, adhering to DIP by depending on abstractions rather than concrete implementations.

Summary of Lessons from Case Studies

Maintainability: Refactoring large, monolithic classes into smaller, focused ones increases maintainability and makes the code easier to understand and modify.

Scalability: Systems designed with SOLID principles can adapt more easily to new requirements and changes in functionality.

Flexibility: By adhering to principles like DIP and OCP, software becomes more flexible and can integrate new features with minimal impact on existing code.